

AgriBot-Agent

Progetto di Introduzione all'Intelligenza Artificiale

Tommaso Lauria

Vannucci Lorenzo

2026

Indice

1. Introduzione e Obiettivi
2. Architettura Generale del Sistema
3. Struttura del Progetto e File Utilizzati
4. Fase 1 — Addestramento della Rete Neurale (train.py)
 - 4.1. Caricamento del Dataset EMNIST
 - 4.2. Generazione delle Celle Vuote (classe ".")
 - 4.3. Architettura della CNN
 - 4.4. Training e Early Stopping
 - 4.5. Grafici delle Prestazioni
5. Fase 2 — Ritaglio della Griglia dall'Immagine (grid_cell_extractor.py)
 - 5.1. Caricamento e Binarizzazione
 - 5.2. Rilevamento del Contorno Esterno
 - 5.3. Ordinamento dei Vertici e Trasformazione Prospettica
 - 5.4. Individuazione degli Intervalli delle Celle
 - 5.5. Pulizia delle Singole Celle (clean_cell)
 - 5.6. Estrazione Finale e Restituzione
6. Fase 3 — Riconoscimento e Costruzione della Mappa (main.py)
 - 6.1. Classificazione CNN delle Celle
 - 6.2. Costruzione della Griglia Logica
7. Fase 4 — Definizione del Problema di Ricerca (AgriBotProblem)
 - 7.1. Rappresentazione dello Stato
 - 7.2. Azioni Possibili
 - 7.3. Funzione di Transizione
 - 7.4. Test di Obiettivo
8. Fase 5 — Algoritmi di Ricerca e Euristiche
 - 8.1. Uniform Cost Search (UCS)
 - 8.2. A* con Euristica Manhattan
 - 8.3. A* con Euristica Max Pairwise Distance
 - 8.4. Confronto tra gli Algoritmi
9. Fase 6 — Visualizzazione della Soluzione

1. Introduzione e Obiettivi

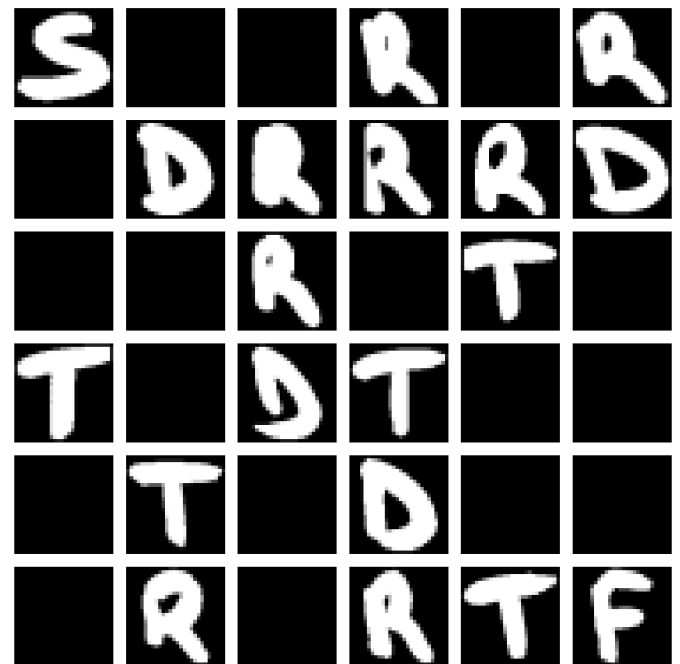
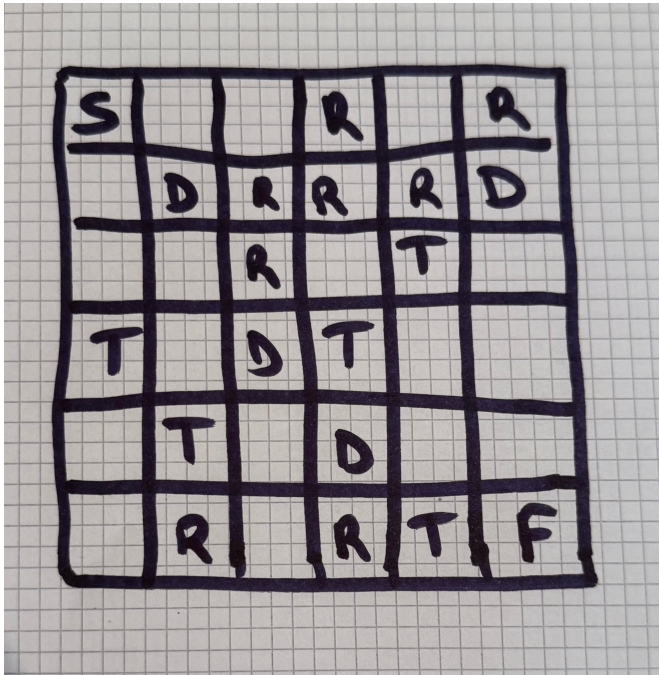
Il nostro AgriBot è un agente progettato per operare in un ambiente agricolo simulato. L'ambiente è rappresentato da una griglia $N \times N$ disegnata su carta (o generata al computer) e fotografata. Il sistema deve:

- Acquisire l'immagine della griglia e ritagliare automaticamente ogni singola cella attraverso una fase di cropping dell'immagine.
- Riconoscere il contenuto di ciascuna cella (lettere D, F, R, S, T, V oppure cella vuota) mediante una rete neurale convoluzionale (CNN) addestrata sul dataset EMNIST.
- Costruire una rappresentazione dell'ambiente (mappa) a partire dalle predizioni della rete.
- Lavoro sulla mappa con algoritmi di ricerca per trovare il percorso migliore. Usiamo algoritmi come UCS e A*.
- Visualizzare graficamente la soluzione trovata, mostrando passo dopo passo le azioni del robot sulla mappa.

Ogni cella della griglia rappresenta un elemento dell'ambiente agricolo:

Simbolo	Significato
S (Start)	Posizione iniziale del robot.
F (Finish)	Posizione finale che il robot deve raggiungere dopo aver completato tutti i compiti.
R (Rock)	Ostacolo invalicabile; il robot non può attraversare questa cella.
D (Dry Plant)	Pianta secca che necessita di 1 unità d'acqua per essere irrigata.
V (Very Dry Plant)	Pianta molto secca che necessita di 2 unità d'acqua.
T (Tank)	Stazione di rifornimento; il robot può ricaricare il serbatoio d'acqua.
. (Vuota)	Terreno libero percorribile.

L'obiettivo del robot è irrigare tutte le piante (D e V) gestendo un serbatoio d'acqua con capacità limitata, e infine raggiungere la cella di arrivo F con il costo minimo possibile.



*Esempio di griglia disegnata su carta e relativa
mappa logica riconosciuta*

Si raggiunge quindi uno stato goal nel momento in cui tutte le piante sono state irrigate e l'AgriBot raggiunge la posizione finale.

2. Architettura Generale del Sistema

Il progetto segue un andamento che si può suddividere in 4 macro-aree:

1. Addestramento: si allena una CNN sul dataset EMNIST Balanced, filtrato sulle sole classi di interesse (D, F, R, S, T, V) a cui si aggiunge una classe per le celle vuote. Il modello addestrato viene salvato su disco (agribot_model.keras).
2. Visione: l'immagine della griglia viene processata con OpenCV per trovare il contorno esterno, correggere la prospettiva, e ritagliare ogni singola cella in un'immagine 28x28 (per la lettura della CNN addestrata con EMNIST) pixel in scala di grigi.
3. Classificazione: ogni cella ritagliata viene passata alla CNN che predice il carattere contenuto. Si costruisce così la mappa logica (matrice di stringhe).
4. Pianificazione e Ricerca: la mappa logica viene usata per iniziare un problema di ricerca nello spazio degli stati. Si applicano UCS e A* (con due euristiche diverse) per trovare il piano d'azione ottimale.

3. Struttura del Progetto e File Utilizzati

Il progetto è organizzato nei seguenti file principali:

train.py: Script per l'addestramento della rete neurale convolutiva (CNN). Scarica il dataset EMNIST Balanced, filtra le classi di interesse, genera celle vuote sintetiche per la classe ".", costruisce il modello CNN, lo addestra con early stopping e salva il modello finale come agribot_model.keras. Produce inoltre il grafico delle prestazioni (accuracy e loss).

grid_cell_extractor.py: Modulo di Computer Vision. Contiene la funzione crop() che riceve il percorso di un'immagine e le dimensioni della griglia (righe x colonne). Rileva automaticamente il contorno della griglia, corregge la prospettiva con una trasformazione proiettiva (warp), individua le linee della griglia tramite apertura morfologica, e ritaglia ogni singola cella producendo immagini 28x28 pronte per la classificazione.

main.py: File principale del progetto. Integra tutte le componenti: (1) chiama crop() per ritagliare le celle dall'immagine, (2) usa la CNN per classificare ogni cella e costruire la mappa logica, (3) definisce la classe AgriBotProblem che modella il problema di ricerca, (4) esegue UCS e A* con due euristiche (Manhattan e Max Pairwise Distance), (5) visualizza graficamente la soluzione passo dopo passo.

agribot_model.keras: File del modello CNN pre-addestrato. Viene generato da train.py e caricato da main.py per la classificazione delle celle.

4. Fase 1 — Addestramento della Rete Neurale (train.py)

4. Fase 1 — Addestramento della Rete Neurale (train.py)

4.1. Caricamento del Dataset EMNIST

Il dataset utilizzato è EMNIST Balanced, una versione estesa di MNIST che include sia cifre sia lettere dell'alfabeto. Viene scaricato automaticamente tramite la libreria `tensorflow_datasets`.

Visto che per il progetto basta utilizzare solo alcune lettere, viene applicato un filtro: si selezionano esclusivamente le classi corrispondenti alle lettere D, F, R, S, T e V. La mappatura tra indici EMNIST e le classi interne utilizzate è definita in questo modo:

```
TARGET_MAP = {13: 0, 15: 1, 27: 2, 28: 3, 29: 4, 31: 6}
CLASSES = ['D', 'F', 'R', 'S', 'T', '.', 'V']
```

Ogni immagine EMNIST è in formato 28x28 pixel in scala di grigi. Le immagini vengono trasposte (**`np.transpose`**) perché EMNIST le memorizza ruotate rispetto alla convenzione standard.

4.2. Generazione delle Celle Vuote (classe ".")

Per addestrare la rete a riconoscere correttamente le celle vuote (dove non è presente alcun simbolo), vengono generate artificialmente delle immagini di sfondo. La quantità di questi dati è pari a un quinto del dataset originale. Per garantire che il modello sia robusto e non confonda piccole imperfezioni con dei caratteri, le immagini sono divise in due gruppi: metà sono completamente nere (sfondo ideale), mentre l'altra metà contiene un rumore casuale di bassa intensità. In questo modo, la rete impara a classificare come 'vuoto' (classe 5) anche le caselle che presentano leggeri disturbi o granulosità, tipici delle scansioni reali.

```
n_noise = len(X) // 5
x_noise = []

for i in range(n_noise):
    img = np.zeros((28, 28), dtype=np.uint8)
    if i % 2 == 0:
        noise = np.random.randint(0, 50, (28, 28), dtype=np.uint8)
        img = cv2.add(img, noise)
    x_noise.append(img)
```

A queste immagini viene assegnata la label 5, che corrisponde alla classe "." (cella vuota) nell'array `CLASSES`.

4.3. Architettura della CNN

Il modello è una rete neurale convoluzionale (Convolutional Neural Network) costruita con l'API Sequential di Keras. L'architettura è la seguente:

Strato	Descrizione
Input	28×28 — immagine in scala di grigi.
RandomRotation(0.1)	Data augmentation: rotazione casuale fino a $\pm 10\%$ per rendere il modello invariante a piccole rotazioni.
RandomZoom(0.1)	Data augmentation: zoom casuale fino a $\pm 10\%$ per gestire variazioni di scala.
Conv2D(32, 3×3, ReLU, same)	Primo strato convoluzionale: 32 filtri 3×3 con attivazione ReLU e padding "same" per mantenere le dimensioni.
MaxPooling2D(2×2)	Riduce la dimensione spaziale della metà (da 28×28 a 14×14).
Dropout(0.2)	Spegne casualmente il 20% dei neuroni durante il training per prevenire l'overfitting.
Conv2D(64, 3×3, ReLU, same)	Secondo strato convoluzionale: 64 filtri per catturare pattern più complessi.
MaxPooling2D(2×2)	Riduce ulteriormente (da 14×14 a 7×7).
Dropout(0.2)	Ulteriore regolarizzazione.
Flatten	Appiattisce la mappa 7×7×64 = 3136 valori in un vettore 1D.
Dense(128, ReLU)	Strato fully-connected con 128 neuroni.
Dropout(0.4)	Dropout più aggressivo prima dell'output per massima regolarizzazione.
Dense(7, Softmax)	Strato di output: 7 neuroni (uno per classe) con Softmax per produrre probabilità.

Il modello viene compilato con ottimizzatore Adam, funzione di loss categorical_crossentropy (adatta alla classificazione multi-classe) e metrica accuracy.

4.4. Training e Early Stopping

Il dataset viene suddiviso in 80% training e 20% validation tramite ***train_test_split*** con stratificazione (***stratify=Y***), che garantisce che ogni classe sia rappresentata proporzionalmente in entrambi i set.

Il training viene eseguito per un massimo di 30 epoche con batch size 64. Viene utilizzato un callback EarlyStopping che monitora la validation loss: se questa non migliora per 5 epoche consecutive (***patience=5***), il training si interrompe automaticamente e vengono ripristinati i pesi migliori (***restore_best_weights=True***). Questo evita l'overfitting.

```
early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

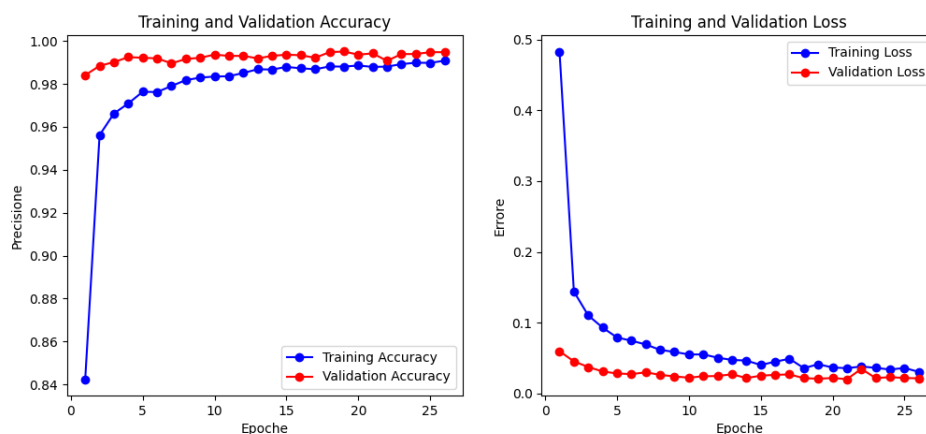
print("Inizio training...")
# Salviamo la storia dell'allenamento nella variabile 'history'
history = model.fit(X_train, Y_train, validation_data=(X_val, Y_val), epochs=30, batch_size=64, callbacks=[early_stop])
model.save("agribot_model.keras")
```

4.5. Grafici delle Prestazioni

Terminato il training, la funzione ***plot_history()*** genera due grafici affiancati:

- **Training Accuracy vs Validation Accuracy:** mostra come la precisione evolve durante le epoche.
- **Training Loss vs Validation Loss:** mostra come l'errore diminuisce. Se le curve di training e validation divergono, indica overfitting.

I grafici vengono salvati come *statistiche_training.png*.



Grafici di Training Accuracy e Training Loss (*statistiche_training.png*)

5. Fase 2 — Ritaglio della Griglia dall'Immagine (grid_cell_extractor.py)

La funzione principale **crop(filename, n_rows, n_cols)** riceve il percorso di un'immagine e le dimensioni della griglia, e restituisce una lista di tuple (riga, colonna, immagine_28x28) per ogni cella. Il processo si articola nelle seguenti fasi:

5.1. Caricamento e Binarizzazione

L'immagine viene caricata con **cv2.imread()** e convertita in scala di grigi con **cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)**. Successivamente si applica una sogliatura binaria inversa con il metodo di Otsu:

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, negative_img = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV | cv2.THRESH_OTSU)
```

THRESH_BINARY_INV inverte i colori: lo sfondo chiaro diventa nero (0) e le linee/lettere scure diventano bianche (255). **THRESH_OTSU** calcola automaticamente la soglia ottimale analizzando l'istogramma dell'immagine, senza bisogno di specificare un valore manuale.

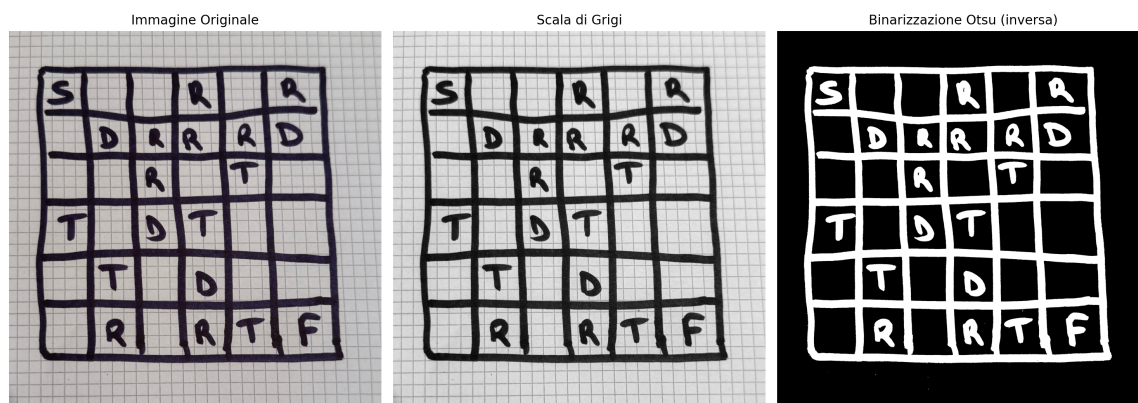


Immagine originale e risultato della binarizzazione con Otsu

5.2. Rilevamento del Contorno Esterno

Sull'immagine binarizzata si cercano i contorni con **cv2.findContours()**. Il flag **RETR_EXTERNAL** indica di considerare solo i contorni esterni (il perimetro della griglia), ignorando i contorni interni (lettere, linee interne). **CHAIN_APPROX_SIMPLE** comprime i segmenti rettilinei memorizzando solo i punti estremi, risparmiando memoria.

```
find_contours, _ = cv2.findContours(negative_img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
find_biggest_contours = sorted(find_contours, key=cv2.contourArea, reverse=True)
```

I contorni trovati vengono ordinati per area in ordine decrescente: il più grande corrisponde al bordo esterno della griglia.

5.3. Ordinamento dei Vertici e Trasformazione Prospettica

Dal contorno più grande si calcola il rettangolo minimo ruotato (`minAreaRect`), i cui 4 vertici vengono estratti con la funzione di OpenCV `boxPoints`. Questi vertici rappresentano gli angoli della griglia nell'immagine originale, anche se questa è stata fotografata storta o in maniera non simmetrica.

```
corners = cv2.boxPoints(cv2.minAreaRect(c)).astype(np.float32)
```

I vertici vengono poi ordinati in senso preciso (Top-Left, Top-Right, Bottom-Right, Bottom-Left) dalla funzione `order_points()`, che utilizza la somma ($x+y$) e la differenza ($x-y$) delle coordinate:

- Il vertice con la somma $x+y$ minima è il Top-Left.
- Il vertice con la somma $x+y$ massima è il Bottom-Right.
- Il vertice con la differenza $x-y$ massima è il Top-Right.
- Il vertice con la differenza $x-y$ minima è il Bottom-Left.

Con i vertici sorgente (`src`) e quelli di destinazione (`dst` = quadrato 1000×1000), si calcola

```
dst = np.float32([[0,0],[DIM-1,0],[DIM-1,DIM-1],[0,DIM-1]])
warped = cv2.warpPerspective(gray, cv2.getPerspectiveTransform(src, dst), (DIM, DIM))
```

la matrice di trasformazione prospettica e si applica il warp:

Il risultato è un'immagine perfettamente raddrizzata di 1000×1000 pixel, indipendentemente dall'angolo o dalla distorsione della foto originale. Sull'immagine "warped" si applica nuovamente la soglia di Otsu (per binarizzare) seguita da un'apertura morfologica con kernel 3×3 per eliminare il rumore di fondo.

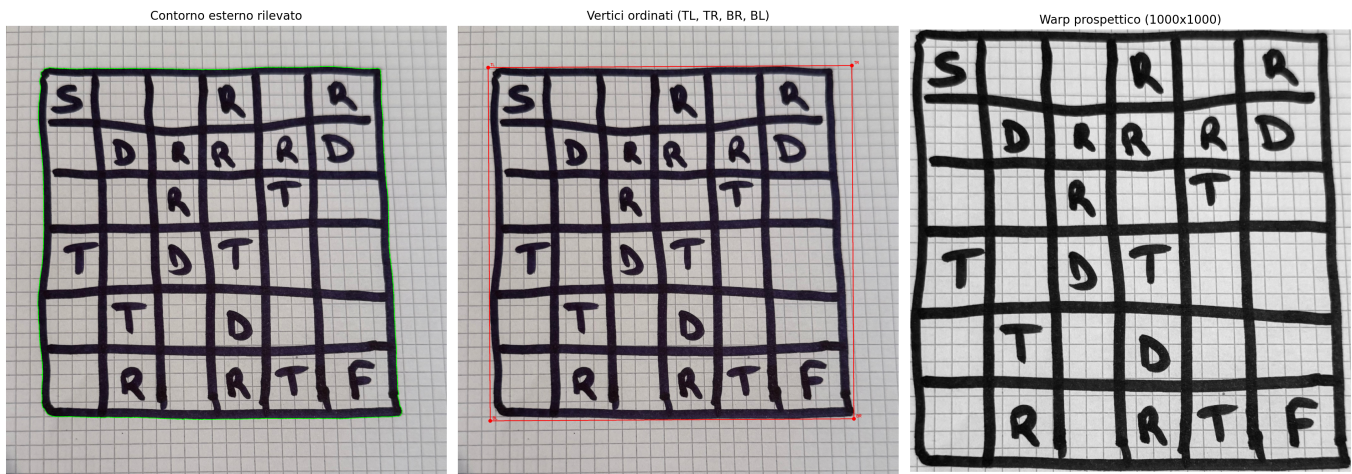


Immagine originale, vertici rilevati e risultato del warp prospettico a 1000x1000

5.4. Individuazione degli Intervalli delle Celle

Questa è la fase più delicata: bisogna individuare dove iniziano e finiscono le righe e le colonne della griglia all'interno dell'immagine "warpat". Si utilizza un approccio basato sull'apertura morfologica per isolare esclusivamente le linee della griglia, eliminando le lettere.

Apertura Morfologica per l'Estrazione delle Linee

L'apertura morfologica (MORPH_OPEN) è composta da due operazioni consecutive: un'erosione seguita da una dilatazione. L'idea chiave è utilizzare un kernel rettangolare molto allungato:

- Per trovare le linee orizzontali: si usa un kernel largo (es. 150 pixel) e alto 1 pixel. L'erosione elimina tutte le strutture bianche che non sono almeno larghe 150 pixel in modo continuo. Le lettere (larghe meno di 150 pixel) vengono cancellate. Le linee della griglia (larghe ~1000 pixel, cioè l'intera immagine) sopravvivono. La dilatazione successiva riporta le strutture sopravvissute alla loro dimensione originale.
- Per trovare le linee verticali: si usa un kernel alto (es. 150 pixel) e largo 1 pixel, con lo stesso principio.
- Nel codice sottostante possiamo vedere che prendiamo il massimo tra il 60% del `expected_size` (ovvero la dimensione warp / la `n` della matrice) e 40.

```
kernel_len = max(int(expected_size * 0.6), 40)
if direction == 'h':
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (kernel_len, 1))
    lines_only = cv2.morphologyEx(warped_img, cv2.MORPH_OPEN, kernel)
    proj = lines_only.sum(axis=1).astype(float)
else:
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (1, kernel_len))
    lines_only = cv2.morphologyEx(warped_img, cv2.MORPH_OPEN, kernel)
    proj = lines_only.sum(axis=0).astype(float)
```


Dopo l'apertura morfologica, l'immagine **binaria lines_only** contiene solo le linee della griglia, senza nessuna lettera. La proiezione (somma lungo un asse) produce un vettore 1D di 1000 valori: i valori sono alti dove c'è una linea e zero dove non c'è.

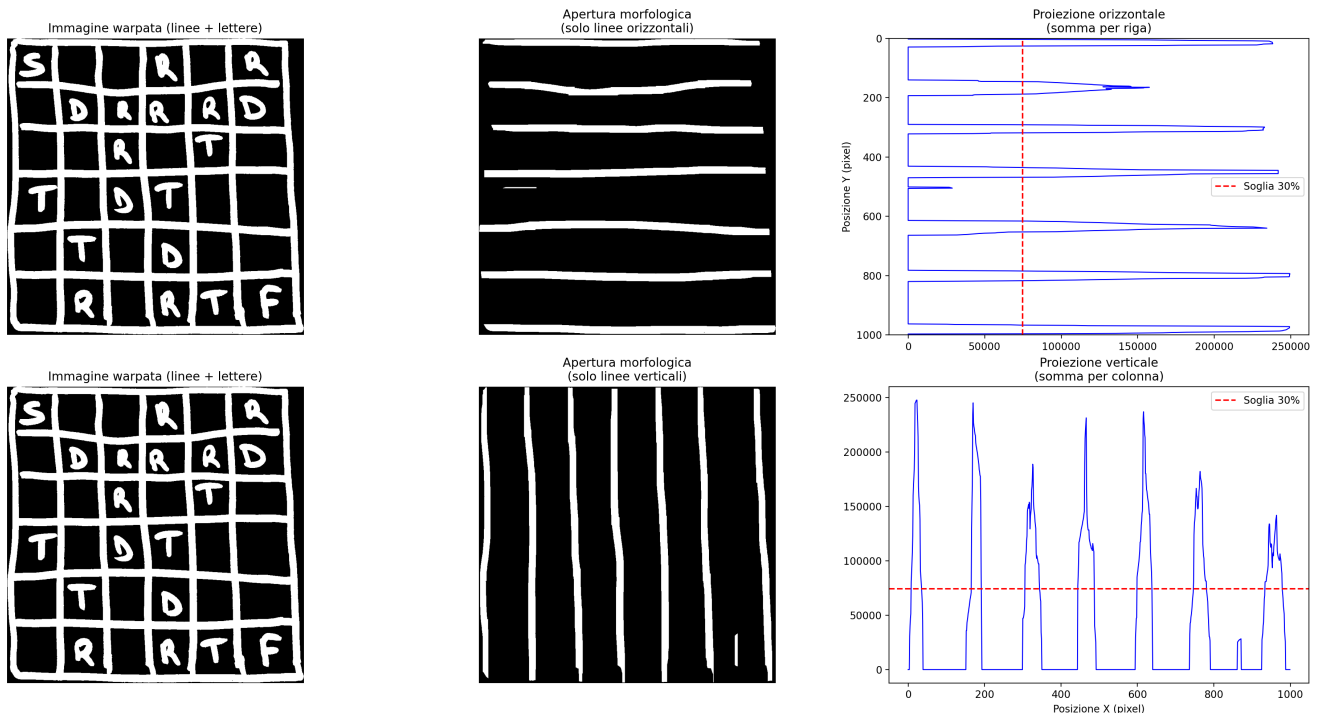


Immagine dopo apertura morfologica (solo linee) e proiezione risultante

Ricerca dei Segmenti di Linea

Sulla proiezione si applica una soglia al 30% del valore massimo per ottenere un array booleano (True = linea, False = no linea). Per trovare l'inizio e la fine di ogni segmento contiguo di True, si usa il seguente metodo:

```
# 2. Soglia per individuare dove ci sono linee nella proiezione
threshold = proj.max() * 0.3
is_line = proj > threshold

# 3. Trova segmenti contigui di linea (inizio e fine di ciascuno)
padded = np.concatenate([[False], is_line, [False]])
diff = np.diff(padded.astype(int))
line_starts = np.where(diff == 1)[0]
line_ends = np.where(diff == -1)[0]
```


np.diff() calcola la differenza tra elementi consecutivi: dove il valore passa da 0 a 1 (diff == 1) si ha l'inizio di una linea; dove passa da 1 a 0 (diff == -1) si ha la fine. L'aggiunta di padding con False all'inizio e alla fine della sequenza di True e false garantisce che anche le linee ai bordi vengano rilevate.

Costruzione dei Gap (Intervalli delle Celle)

Una volta note le posizioni esatte di tutte le linee, le celle corrispondono allo spazio tra linee consecutive. Si costruisce una lista di confini alternati [0, inizio_linea_1, fine_linea_1, inizio_linea_2, fine_linea_2, ..., 1000] e si estraggono i gap alle posizioni pari:

```
boundaries = [0]
for s, e in zip(line_starts, line_ends):
    boundaries.append(int(s))
    boundaries.append(int(e))
boundaries.append(DIM)

gaps = []
for i in range(0, len(boundaries) - 1, 2):
    g_start = boundaries[i] + MARGIN
    g_end = boundaries[i + 1] - MARGIN
    if g_end > g_start and (g_end - g_start) > expected_size * 0.15:
        gaps.append((g_start, g_end))
```

Il MARGIN di 2 pixel restringe leggermente ogni cella per evitare di includere pixel residui della linea. I gap troppo piccoli (meno del 15% della dimensione attesa) vengono scartati — tipicamente sono gli spazi tra il bordo dell'immagine e il bordo esterno della griglia. Infine, tra tutti i gap validi, si selezionano gli n più vicini alla dimensione attesa di una cella (DIM/n) e si riordinano per posizione.

5.5. Pulizia delle Singole Celle (clean_cell)

Ogni cella ritagliata dalla griglia viene passata alla funzione **clean_cell()**, che la pulisce e la prepara per la classificazione CNN. I passaggi sono:

1. Bordo nero: si imposta a 0 la prima e l'ultima riga e colonna della cella per eliminare eventuali residui delle linee della griglia che potrebbero essere rimasti ai bordi.
2. Connected Components: si usa **cv2.connectedComponentsWithStats()** che analizza l'immagine binaria e raggruppa i pixel bianchi adiacenti in "isole". Ogni isola riceve un'etichetta (label) e vengono calcolate le statistiche: posizione (x, y), dimensioni (w, h) e area.
3. Selezione dell'isola principale: si seleziona la componente connessa con l'area maggiore (escludendo lo sfondo nero, che è la componente 0). Questa dovrebbe essere la lettera.
4. Filtri di validazione: si applicano diversi controlli per escludere falsi positivi:

- Se l'area è inferiore a 50 pixel, è rumore → cella vuota.
- Se il rapporto w/h è maggiore di 5 o inferiore a 0.2, è una forma troppo allungata → cella vuota.
- Se l'oggetto è molto sottile e lungo (larghezza < 6px e altezza > 50% della cella), è un residuo di linea verticale della griglia → cella vuota. Analogamente per le linee orizzontali.

```
if area < 50:
    return np.zeros((28, 28), np.uint8)
rapporto = w / h
if rapporto > 5 or rapporto < 0.2:
    return np.zeros((28, 28), np.uint8)

H_cell, W_cell = cell.shape
is_vertical_line = w < 6 and (h > H_cell * 0.5)
is_horizontal_line = h < 6 and (w > W_cell * 0.5)
if is_vertical_line or is_horizontal_line:
    return np.zeros((28, 28), np.uint8)
```

5. Ritaglio e centratura: la lettera viene ritagliata dal suo bounding box, inserita in un quadrato (con padding nero simmetrico) e ridimensionata a 28×28 pixel con interpolazione **cv2.INTER_AREA**, per evitare di avere linee troppo poco dense di pixel

clean_cell() passo passo — Cella (1,1)

1. Cella grezza
(ritagliata dal warp)



2. Bordo nero 3px
(rimuove residui griglia)



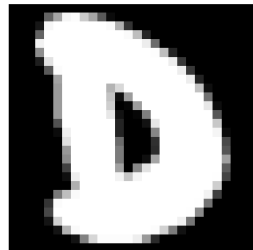
3. Componenti connesse
(verde = più grande, area=3208)



4. Ritaglio lettera
(bounding box 64×77)



5. Risultato finale
(centrata 28×28)



Esempio di cella prima e dopo la pulizia con clean_cell

5.6. Estrazione Finale e Restituzione

La funzione **crop()** itera su tutte le combinazioni di righe e colonne, ritaglia ogni cella dall'immagine warpata usando gli intervalli trovati, la pulisce con **clean_cell()**, e restituisce una lista di tuple (i, j, immagine_28x28) dove i è l'indice di riga, j l'indice di colonna e immagine_28x28 è l'immagine pronta per la CNN.

```
cells = []
for i, r in enumerate(rows[:n_rows]):
    for j, c in enumerate(cols[:n_cols]):
        cells.append((i, j, clean_cell(warped[r[0]:r[1], c[0]:c[1]])))
return cells
```

6. Fase 3 — Riconoscimento e Costruzione della Mappa (main.py)

6.1. Classificazione CNN delle Celle

Il file `main.py` inizia caricando il modello CNN pre-addestrato e ritagliando le celle dall'immagine con `crop()`. Ogni cella 28×28 viene normalizzata (valori tra 0 e 1), ridimensionata a (1, 28, 28, 1) per il formato batch richiesto da Keras, e passata al modello per la predizione.

```
model = tf.keras.models.load_model("agribot_model.keras")
cells = crop("agribot_map_L1.png", N_ROWS, N_COLS)
for i, j, cell_img in cells:
    # Preprocessing img
    normalized_cell = cell_img.astype("float32") / 255.0
    normalized_cell = normalized_cell.reshape(1, 28, 28, 1)

    # Predizione
    prediction = model.predict(normalized_cell, verbose=0) # verbose=0 per pulire output
    idx = np.argmax(prediction)
    char = LABELS[idx] # LABELS = ['D', 'F', 'R', 'S', 'T', '.', 'V']
```

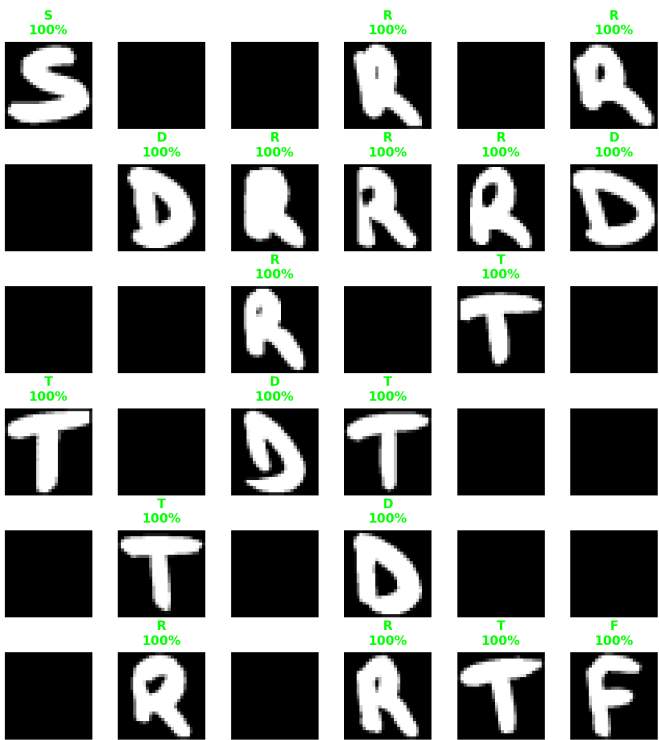
`model.predict()` restituisce un array di 7 probabilità (una per classe). `np.argmax()` seleziona l'indice con la probabilità più alta, che viene convertito nel carattere corrispondente tramite l'array `LABELS`.

6.2. Costruzione della Griglia Logica

I caratteri riconosciuti vengono organizzati in una matrice (lista di liste) `grid_map`, dove ogni riga interna contiene i caratteri delle `N_COLS` colonne. Questa matrice rappresenta la mappa logica dell'ambiente e viene passata al costruttore di `AgriBotProblem`.

```
grid_map = []
row = []
for i, j, cell_img in cells:
    # ... Predizione ...
    row.append(char)
    # Se abbiamo finito la riga corrente (colonna == ultima colonna)
    if j == N_COLS - 1:
        grid_map.append(row)
        row = []
```

Classificazione CNN → Mappa Logica



Mappa Logica (grid_map)

S	.	.	R	.	R
.	D	R	R	R	D
.	.	R	.	T	.
T	.	D	T	.	.
.	T	.	D	.	.
.	R	.	R	T	F

S = Start F = Finish R = Roccia D = Pianta secca V = Molto secca T = Stazione . = Vuota

7. Fase 4 — Definizione del Problema di Ricerca (AgriBotProblem)

La classe AgriBotProblem eredita dalla classe Problem della libreria AIMA. Questa modellazione consente di utilizzare direttamente gli algoritmi di ricerca (**UCS**, **A***) forniti dalla libreria.

7.1. Rappresentazione dello Stato

Lo stato del robot è rappresentato dalla tupla:

- (position, water, dry, very_dry)

Dove:

- **position**: un intero che rappresenta la cella corrente del robot (indice lineare $rxn+c$).
- **water**: il livello corrente del serbatoio d'acqua (intero, da 0 a max_water).
- **dry**: un frozenset contenente gli indici delle piante D ancora da irrigare.
- **very_dry**: un frozenset contenente gli indici delle piante V ancora da irrigare.

L'uso di frozenset è necessario perché gli stati devono essere hashable per poter essere inseriti negli insiemi "esplorati" degli algoritmi di ricerca. Ogni volta che una pianta viene irrigata, si crea un nuovo frozenset senza quella pianta.

La conversione da coordinate (riga, colonna) a indice lineare avviene con la formula **index = riga × n + colonna**, dove n è la dimensione della griglia.

Ad esempio, in una griglia 4×4, la cella (1, 2) corrisponde all'indice $1 \times 4 + 2 = 6$.

7.2. Azioni Possibili

Il metodo actions(state) restituisce la lista delle azioni valide nello stato corrente. Le azioni sono generate dinamicamente in base alla posizione, alle risorse e agli elementi circostanti:

Azione	Descrizione e Condizioni
UP, DOWN, LEFT, RIGHT	Movimento nelle 4 direzioni cardinali. Un'azione di movimento è valida solo se la cella di destinazione è dentro la griglia e non contiene una roccia (R). Costo: 1.
WATER	Irrigazione della pianta nella cella corrente. Valida solo se il robot è su una cella D (richiede 1 unità d'acqua) o V (richiede 2 unità d'acqua) e ha acqua sufficiente. Costo: 1.
REFILL	Ricarica del serbatoio. Valida solo se il robot è su una stazione T e il serbatoio non è già pieno. Riporta l'acqua al massimo. Costo: 1.

7.3. Funzione di Transizione

Il metodo **result(self, state, action)** restituisce il nuovo stato risultante dall'applicazione di un'azione. Per i movimenti, la posizione viene aggiornata sommando il delta corrispondente alla direzione (-n per UP, +n per DOWN, -1 per LEFT, +1 per RIGHT). Per WATER, la pianta viene rimossa dal set corrispondente e l'acqua viene decrementata. Per REFILL, l'acqua torna al massimo.

```
def result(self, state, action):
    position, water, dry, very_dry = state
    delta = {'UP': -self.n, 'DOWN': self.n, 'LEFT': -1, 'RIGHT': 1}
    if action == "WATER" and position in dry:
        new_dry=dry.difference({position})
        new_water=water-1
        return (position, new_water, new_dry,very_dry)
    if action == "WATER" and position in very_dry:
        new_very_dry=very_dry.difference({position})
        new_water=water-2
        return (position, new_water, dry,new_very_dry)
    if action == "REFILL":
        return (position, self.max_water, dry,very_dry)

    new_position= position + delta[action]
    return (new_position,water,dry,very_dry)
```

7.4. Test di Obiettivo

Il metodo **goal_test(state)** verifica se lo stato corrente è un obiettivo. La condizione è che tutte le piante siano state irrigate (entrambi i set dry e very_dry sono vuoti) e il robot si trovi sulla cella finale F.

```
def goal_test(self, state):
    position, water, dry, very_dry = state
    if (len(dry)==0) and (len(very_dry)==0) and position == self.finish_position:
        return True
    else:
        return False
```

8. Fase 5 — Algoritmi di Ricerca e Euristiche

Il progetto confronta tre approcci di ricerca.

8.1. Uniform Cost Search (UCS)

UCS è un algoritmo di ricerca non informata che espande sempre il nodo con il costo di percorso più basso. Garantisce di trovare la soluzione ottimale (costo minimo) ma esplora

molti nodi inutili perché non ha informazioni sulla direzione dell'obiettivo. Viene usato come base per valutare l'efficacia delle euristiche.

8.2. A* con Euristica Manhattan (**h_manhattan**)

A* è un algoritmo di ricerca informata che utilizza una funzione di valutazione $f(n) = g(n) + h(n)$, dove $g(n)$ è il costo dal nodo iniziale al nodo n , e $h(n)$ è una stima euristica del costo dal nodo n all'obiettivo.

L'euristica Manhattan (**h_manhattan**) considera tre casi in base allo stato corrente:

- **A:** Tutte le piante irrigate: restituisce la distanza Manhattan dalla posizione corrente alla cella di arrivo F. Non ci sono più compiti, bisogna solo raggiungere la fine.
- **B:** Ci sono ancora piante e il robot ha acqua: restituisce la distanza Manhattan verso la pianta più vicina. Il prossimo passo è irrigare.
- **C:** Ci sono ancora piante ma il robot è senza acqua: restituisce la distanza Manhattan verso la stazione di rifornimento T più vicina. Bisogna prima ricaricare.

```
def h_manhattan(self,node):
    position, water, dry, very_dry = node.state
    all_plants = dry.union(very_dry)
    if len(all_plants)==0:
        return self.cal_manhattan(position,self.finish_position)

    # Caso: ho acqua a sufficienza (almeno 1)
    # Considero tutte le piante come target validi per l'euristica
    if water > 0:
        plant_dists=[]
        for index in all_plants:
            plant_dists.append(self.cal_manhattan(position,index))
        return min(plant_dists)

    # Caso: sono a secco
    if len(all_plants)>0 and water==0:
        station_dist=[]
        for index in self.station:
            station_dist.append(self.cal_manhattan(position,index))
        return min(station_dist)
```

Questa euristica è ammissibile (non sovrastima mai il costo reale) e quindi A* con **h_manhattan** garantisce l'ottimalità. Tuttavia sottostima significativamente il costo in scenari complessi con molte piante sparse, perché considera solo l'obiettivo più vicino ignorando tutti gli altri.

8.3. A* con Euristiche Max Pairwise Distance (h_max_pairwise_Distance)

Questa euristica più sofisticata aggiunge alla stima il "diametro" del set di piante rimaste, cioè la massima distanza Manhattan tra qualsiasi coppia di piante. L'intuizione è: anche dopo aver raggiunto la pianta più vicina, il robot dovrà comunque percorrere almeno la distanza tra le due piante più lontane tra loro.

```
def h_max_pairwise_Distance(self,node):
    position, water, dry, very_dry = node.state

    all_plants = dry.union(very_dry)

    if len(all_plants)==0:
        return self.cal_manhattan(position,self.finish_position)

    max_internal_distance = 0
    plants_list=list(all_plants)
    for i in range(len(plants_list)):
        for j in range(i + 1, len(plants_list)):
            dist = self.cal_manhattan(plants_list[i], plants_list[j])
            if dist > max_internal_distance:
                max_internal_distance = dist

    if(water==0):
        station_dist=[]
        for index in self.station:
            station_dist.append(self.cal_manhattan(position,index))
        return min(station_dist)+max_internal_distance

    if water > 0:
        plant_dists=[]
        for index in all_plants:
            plant_dists.append(self.cal_manhattan(position,index))
        most_close_plants=min(plant_dists)
        return max_internal_distance+most_close_plants
```

8.4. Confronto tra gli Algoritmi

Il progetto esegue tutti e tre gli algoritmi sulla stessa mappa e confronta le statistiche tramite **InstrumentedProblem** della libreria **AIMA**, che **conta automaticamente il numero di nodi esplorati e goal test effettuati**.

Algoritmo	Euristica	Nodi Esplorati	Costo Soluzione
UCS	Nessuna	2731	914
A*	Manhattan	2274	748
A*	Max Pairwise Distance	1781	575

Il costo della soluzione è identico per tutti e tre gli algoritmi (ottimalità), ma il numero di nodi esplorati diminuisce progressivamente da UCS ad A* con Max Pairwise Distance, dimostrando il valore di un'euristica più informativa.

```

Avvio ricerca UCS
Nodi esplorati da UCS: 2731
Goal test effettuati: 914
Costo: 21
passi: (21):
['RIGHT', 'DOWN', 'WATER', 'DOWN', 'DOWN', 'RIGHT', 'WATER', 'RIGHT', 'REFILL', 'DOWN', 'WATER', 'UP', 'UP', 'RIGHT', 'RIGHT', 'UP', 'WATER', 'DOWN', 'DOWN', 'DOWN', 'DOWN']

Avvio ricerca A* con euristica max_pairwise_Distance...
Costo Totale: 21
Numero passi: 21
Azioni: ['RIGHT', 'DOWN', 'WATER', 'DOWN', 'DOWN', 'RIGHT', 'WATER', 'RIGHT', 'REFILL', 'DOWN', 'WATER', 'UP', 'UP', 'RIGHT', 'RIGHT', 'UP', 'WATER', 'DOWN', 'DOWN', 'DOWN', 'DOWN']
Nodi esplorati da A*: 1781
Goal test effettuati: 575

Avvio ricerca A* con euristica Manhattan...
Costo Totale: 21
Numero passi: 21
Azioni: ['RIGHT', 'DOWN', 'WATER', 'DOWN', 'DOWN', 'RIGHT', 'WATER', 'RIGHT', 'REFILL', 'DOWN', 'WATER', 'UP', 'UP', 'RIGHT', 'RIGHT', 'UP', 'WATER', 'DOWN', 'DOWN', 'DOWN', 'DOWN']
Nodi esplorati da A*: 2274
Goal test effettuati: 748

```

9. Fase 6 — Visualizzazione della Soluzione

Il file main.py include due modalità di visualizzazione della soluzione trovata:

Visualizzazione Testuale (print_grid)

La funzione **print_grid()** stampa nel terminale una rappresentazione colorata dello stato corrente della griglia, giallo per il robot (B), verde per le piante (D, V), rosso per le rocce (R), blu per le stazioni (T), e magenta per la posizione iniziale (S). Ad ogni step viene mostrata l'azione eseguita e le informazioni sulle risorse (acqua rimanente, piante residue).

Visualizzazione Grafica Interattiva (visualizza_semplice)

La funzione **visualizza_semplice()** crea una finestra Matplotlib interattiva con quattro pulsanti: Play, Pausa, Indietro e Avanti. La griglia viene disegnata come una matrice di celle colorate:

- Grigio: rocce (R)
- Azzurro: stazioni rifornimento (T)
- Oro: cella di arrivo (F)
- Arancione: piante secche (D)
- Rosso: piante molto secche (V)
- Verde: robot (B)

Un timer interno gestisce la riproduzione automatica (Play): ad ogni tick avanza di uno step e ridisegna la griglia. I pulsanti Indietro e Avanti permettono la navigazione manuale. L'utente può così osservare in dettaglio come il robot pianifica ed esegue ogni singola azione.

